

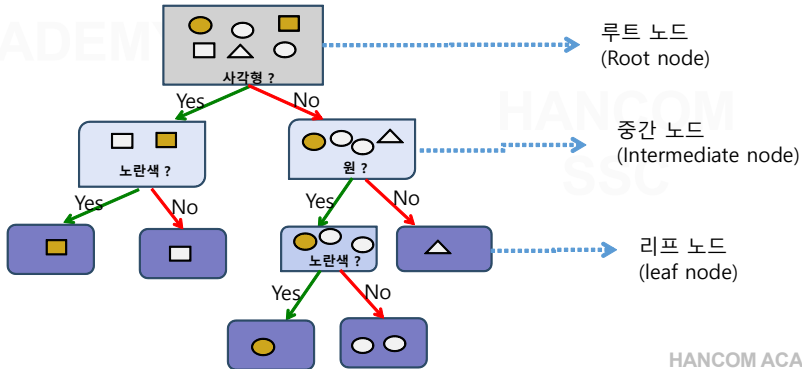
Chapter 04 .scikit-learn 의사결정트리

- 의사결정트리 알고리즘 개념
- 의사결정트리_서울시 행정구역 분류
- 등고선 시각화 활용_결정 경계 표현

Decision Tree(의사결정 트리)

: 의사결정 규칙을 나무구조로 나타내는 전체 데이터를 소집단으로 분류하거나 예측하는 분석 기법

`from sklearn import tree`
`tree.DecisionTreeClassifier()` 클래스 활용



sklearn(사이킷런 : scikit-learn)

- Decision Tree(의사결정 트리) 알고리즘 장/단점

장점	단점
- 모델의 추론 과정을 시각화하기 쉬움 - 학습 및 예측 속도가 빠름 - 데이터 중 중요한 특성이 무엇인지 파악하기 쉬움 - 수치 데이터 및 범주 데이터에 모두 사용 가능 - 특징 변수의 정규화나 표준화가 필요 없음	- 과대 적합되기 쉬움 - 트리깊이, 리프노드 샘플개수 등, 조정해야 할 하이퍼파라미터가 많음

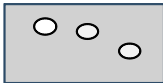
- 데이터 분류(트리 성장 기준) : 불순도가 낮아지는 방향

즉, 모양 or 색 중 어떤 기준으로 분류하여 트리를 성장하는것이 효율적인가?
불순도가 낮아지는 방향으로 분류, 대표적으로 C4.5, CART 알고리즘 활용

* 불순도(Impurity) : 순도의 반대말, 데이터의 섞임 정도



(불순도가 높은 상태)



(순도 100% 상태)

* 불순도(Impurity) 수치화 지표 :

엔트로피(Entropy), 지니계수(Gini index)

- C4.5 : 불순도를 엔트로피로 계산한 알고리즘
- CART : 불순도를 지니계수로 계산한 알고리즘

엔트로피 : 1 이면 불순도 최대(한 범주안에 서로 다른 데이터가 정확히 반반 있다는 뜻)

엔트로피 : 0 이면 불순도는 최소(한 범주안에 하나의 데이터만 있다는 뜻)

C4.5 알고리즘 : Entropy가 낮아지는 방향으로 트리 성장
(엔트로피로 불순도 계산)

* 정보 이득량 : 분할 전 Entropy - 분할 후 Entropy

$$\text{Gain}(T, X) = \text{Entropy}(T) - \text{Entropy}(T, X)$$

- 정보획득량이 크다는 것은 어떠한 속성으로 분할 했을 경우 불순도가 감소했다는 의미

- 가지고 있는 모든 속성(X)에 대해 분할 후 정보 획득량 계산, 이 값이 가장 큰 속성부터 분할의 기준으로 이용

* C4.5알고리즘 활용 기상조건에 따른 경기 참가여부 분류 예

날짜	날씨	온도	습도	바람	참가여부
2021.09.01	맑음	더움	높음	약함	X
2021.09.02	맑음	더움	높음	강함	X
2021.09.03	흐림	더움	높음	약함	O
2021.09.04	비	포근	높음	약함	O
2021.09.05	비	서늘	정상	약함	O
2021.09.06	비	서늘	정상	강함	X
2021.09.07	흐림	서늘	정상	강함	O
2021.09.08	맑음	포근	높음	약함	X
2021.09.09	맑음	서늘	정상	약함	O
2021.09.10	비	포근	정상	약함	O
2021.09.11	맑음	포근	정상	강함	O
2021.09.12	흐림	포근	높음	강함	O
2021.09.13	흐림	더움	높음	약함	O
2021.09.14	비	포근	높음	강함	X

총 : 14 경기

p_i : 한 영역안에 존재하는 데이터 가운데

범주 i 에 속하는 데이터의 비율

m : 한 영역안에 존재하는 범주 개수

1) 분할 전, 목표 속성인 참가여부 에 대한

엔트로피 계산

$$\text{Entropy}(T) = \sum_{i=1}^m -p_i \log_2 p_i$$

$E(\text{경기}) =$

$$-\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) = 0.940$$

실제 모든 데이터는 정수, 실수 데이터로
처리해야 함

clf =

tree.DecisionTreeClassifier(criterion='
entropy',

HANCOM ACADEMY

2) 각 속성에 대해 분할 후 엔트로피 계산

$$\text{Entropy}(T, X) = \sum_{c \in X} p(c) E(c)$$

- X : 선택된 특징
- c : 선택된 특징 하위 노드
- P(c) : 선택된 특징에 의해 생성된 하위 노드에 데이터가 속할 확률
- E(c) : 선택된 특징에 의해 생성된 하위 노드 엔트로피

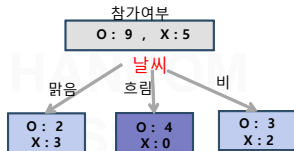
* log 연산

- 1) $\log_2 2^3 = 3$
- 2) $\log_2 1 = 0$
- 3) 분자가 0 이면 0
- 4) $\log_2 \frac{2}{5} = \ln(0.4) / \ln(2)$

2-1) 날씨 속성에 대해 분할 후 엔트로피 계산

$$\text{Entropy}(\text{경기}, \text{날씨}) = \underbrace{p(\text{맑음})}_{5/14} E(\text{경기}, \text{맑음}) + \underbrace{p(\text{흐림})}_{4/14} E(\text{경기}, \text{흐림}) + \underbrace{p(\text{비})}_{5/14} E(\text{경기}, \text{비})$$

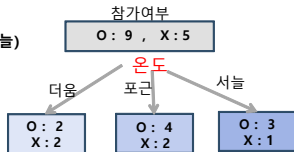
$$\begin{aligned} \text{Entropy}(\text{경기}, \text{날씨}) &= \frac{5}{14} \left(-\frac{2}{5} \log_2 \left(\frac{2}{5} \right) - \frac{3}{5} \log_2 \left(\frac{3}{5} \right) \right) \\ &+ \frac{4}{14} \left(-\frac{4}{4} \log_2 \left(\frac{4}{4} \right) - \frac{0}{4} \log_2 \left(\frac{0}{4} \right) \right) + 0 \\ &+ \frac{5}{14} \left(-\frac{3}{5} \log_2 \left(\frac{3}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) \right) = 0.694 \end{aligned}$$



2-2) 온도 속성에 대해 분할 후 엔트로피 계산

$$\text{Entropy}(\text{경기}, \text{온도}) = p(\text{더움}) * E(\text{경기}, \text{더움}) + p(\text{포근}) * E(\text{경기}, \text{포근}) + p(\text{서늘}) * E(\text{경기}, \text{서늘})$$

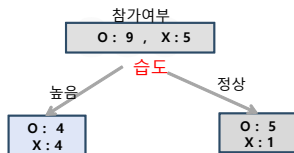
$$\begin{aligned} \text{Entropy}(\text{경기}, \text{온도}) &= \frac{4}{14} \left(-\frac{2}{4} \log_2 \left(\frac{2}{4} \right) - \frac{2}{4} \log_2 \left(\frac{2}{4} \right) \right) \\ &+ \frac{6}{14} \left(-\frac{4}{6} \log_2 \left(\frac{4}{6} \right) - \frac{2}{6} \log_2 \left(\frac{2}{6} \right) \right) \\ &+ \frac{4}{14} \left(-\frac{3}{4} \log_2 \left(\frac{3}{4} \right) - \frac{1}{4} \log_2 \left(\frac{1}{4} \right) \right) = 0.911 \end{aligned}$$



2-3) 습도 속성에 대해 분할 후 엔트로피 계산

$$\text{Entropy}(\text{경기}, \text{습도}) = p(\text{높음}) * E(\text{경기}, \text{높음}) + p(\text{정상}) * E(\text{경기}, \text{정상})$$

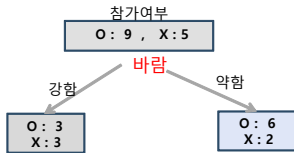
$$\begin{aligned} \text{Entropy}(\text{경기}, \text{습도}) &= \frac{8}{14} \left(-\frac{4}{8} \log_2 \left(\frac{4}{8} \right) - \frac{4}{8} \log_2 \left(\frac{4}{8} \right) \right) \\ &+ \frac{6}{14} \left(-\frac{5}{6} \log_2 \left(\frac{5}{6} \right) - \frac{1}{6} \log_2 \left(\frac{1}{6} \right) \right) = 0.849 \end{aligned}$$



2-4) 바람 속성에 대해 분할 후 엔트로피 계산

$$\text{Entropy}(\text{경기}, \text{바람}) = p(\text{강함}) * E(\text{경기}, \text{강함}) + p(\text{약함}) * E(\text{경기}, \text{약함})$$

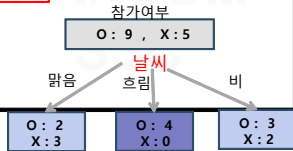
$$\begin{aligned} \text{Entropy}(\text{경기}, \text{바람}) &= \frac{6}{14} \left(-\frac{3}{6} \log_2 \left(\frac{3}{6} \right) - \frac{3}{6} \log_2 \left(\frac{3}{6} \right) \right) \\ &+ \frac{8}{14} \left(-\frac{6}{8} \log_2 \left(\frac{6}{8} \right) - \frac{2}{8} \log_2 \left(\frac{2}{8} \right) \right) = 0.892 \end{aligned}$$



3) 각 속성에 대한 정보 이득(Gain) 계산

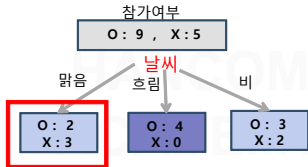
$$\begin{aligned} \text{IG}(\text{경기}, \text{날씨}) &= E(\text{경기}) - E(\text{경기}, \text{날씨}) = 0.94 - 0.694 = 0.246 \\ \text{IG}(\text{경기}, \text{온도}) &= E(\text{경기}) - E(\text{경기}, \text{온도}) = 0.94 - 0.911 = 0.029 \\ \text{IG}(\text{경기}, \text{습도}) &= E(\text{경기}) - E(\text{경기}, \text{습도}) = 0.94 - 0.849 = 0.091 \\ \text{IG}(\text{경기}, \text{바람}) &= E(\text{경기}) - E(\text{경기}, \text{바람}) = 0.94 - 0.892 = 0.048 \end{aligned}$$

Gain이 가장 큰
날씨로 가장 먼저 분류



4) 각 terminal node 에 대해 1)~3) 로직 다시 반복

예) “맑음”으로 분류된 노드인 데이터만 가지고
다시 분할 전 엔트로피 계산



날짜	날씨	온도	습도	바람	참가여부
2021.09.01	맑음	더움	높음	약함	X
2021.09.02	맑음	더움	높음	강함	X
2021.09.08	맑음	포근	높음	약함	X
2021.09.09	맑음	서늘	정상	약함	O
2021.09.11	맑음	포근	정상	강함	O

(총 5개 데이터)

4-1) $E(\text{경기}) = -\frac{2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5} \log_2\left(\frac{3}{5}\right) = 0.971$

4-2) $E(\text{경기}, \text{온도}) = \frac{2}{5} \left(-\frac{2}{2} \log_2\left(\frac{2}{2}\right)\right) + \frac{2}{5} \left(-\frac{1}{2} \log_2\left(\frac{1}{2}\right) - \frac{1}{2} \log_2\left(\frac{1}{2}\right)\right) + \frac{1}{5} \left(-\frac{1}{1} \log_2\left(\frac{1}{1}\right)\right) = 0.4$

4-3) $E(\text{경기}, \text{습도}) = \frac{3}{5} \left(-\frac{3}{3} \log_2\left(\frac{3}{3}\right)\right) + \frac{2}{5} \left(-\frac{2}{2} \log_2\left(\frac{2}{2}\right)\right) = 0$

4-4) $E(\text{경기}, \text{바람}) = \frac{2}{5} \left(-\frac{1}{2} \log_2\left(\frac{1}{2}\right) - \frac{1}{2} \log_2\left(\frac{1}{2}\right)\right) + \frac{3}{5} \left(-\frac{1}{3} \log_2\left(\frac{1}{3}\right) - \frac{2}{3} \log_2\left(\frac{2}{3}\right)\right) = 0.951$

날씨 맑음에 대한 경기확률

온도 더움에 대한 경기확률

온도 포근에 대한 경기확률

온도 서늘에 대한 경기확률

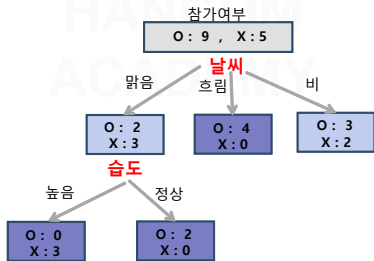
바람 강함에 대한 경기확률

바람 약함에 대한 경기확률

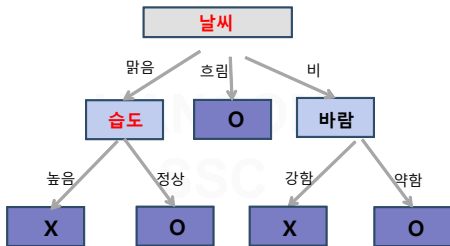
5) 각 속성에 대한 정보 이득(Gain) 계산

$$\begin{aligned} IG(\text{경기, 온도}) &= E(\text{경기}) - E(\text{경기, 온도}) = 0.971 - 0.4 = 0.571 \\ IG(\text{경기, 습도}) &= E(\text{경기}) - E(\text{경기, 습도}) = \mathbf{0.971 - 0 = 0.971} \\ IG(\text{경기, 바람}) &= E(\text{경기}) - E(\text{경기, 바람}) = 0.971 - 0.951 = 0.151 \end{aligned}$$

Gain이 가장 큰
습도 기준으로 분류



최종 결과
(트리)

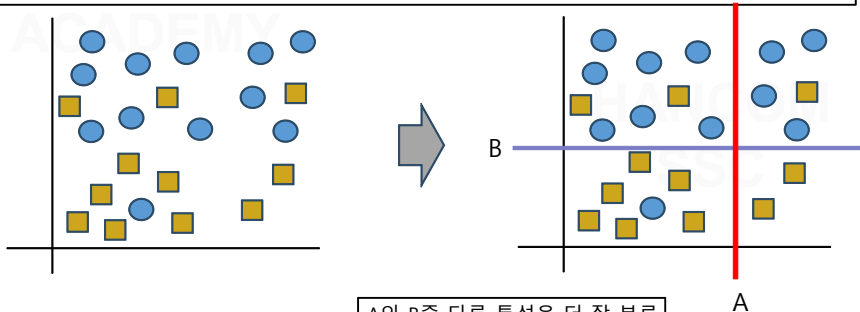


* mydecisiontree_exam.py 샘플코드 참조

* **C4.5** : 엔트로피 계수를 가장 많이 감소시켜주는 변수를 찾아 분리
: 다지분리도 가능하지만 독립 변수가 수치형이면 범위로 그룹화가 되고, **그룹별 엔트로피 계수를 계산해** 이지 분리가 이루어짐

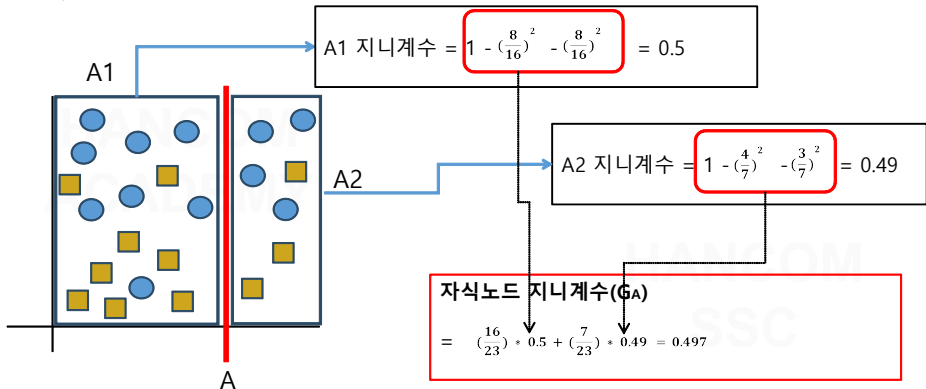
* **CART 알고리즘 : 지니계수(Gini Index) 로 불순도 계산**

- 지니계수 : 불순도 측정 지표, 데이터의 통계적 분산정도를 정량화해서 표현한 값
- 공식 $G(s) = 1 - \sum_{i=1}^c p_i^2$, s : 발생한 사건의 모음, c : 사건의 개수
- 지니 계수가 높을 수록 데이터가 많이 분산되어 있음을 의미 (불순도가 높음)
- 지니 계수를 가장 많이 감소시켜주는 변수를 찾아 분리
- 이지 분류 알고리즘

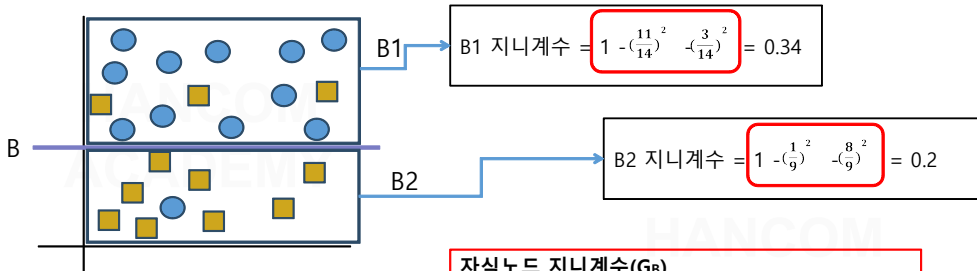


A와 B중 다른 특성을 더 잘 분류
하는지 지니계수 계산 활용

1) A를 기준으로 분할했을 경우 지니 계수



2) B를 기준으로 분할 했을 경우 지니 계수



결론, 지니계수가 낮은
B로 분할하는것 이 효율적

자식노드 지니계수(G_B)

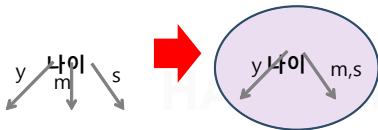
$$= \left(\frac{14}{23}\right) * 0.34 + \left(\frac{9}{23}\right) * 0.2 = 0.28$$

- * 정보 이득(IG) = 부모의 불순도 - 자식노드 불순도
- * 즉, 자식노드 불순도가 낮을 수록 정보 이득이 최대가 됨으로 정보 이득이 최대가 되도록 분할 선택

* CART 알고리즘 활용 나이, 수입, 직장, 신용등급에 따른 집 구입 여부 분류

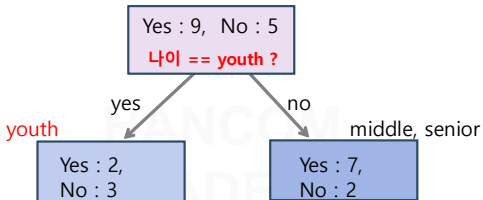
나이	수입	직장	신용	집 구입 여부
youth	high	no	fair	no
youth	high	no	excellent	no
middle	high	no	fair	yes
senior	medium	no	fair	yes
senior	low	yes	fair	yes
senior	low	yes	excellent	no
middle	low	yes	excellent	yes
youth	medium	no	fair	no
youth	low	yes	fair	yes
senior	medium	yes	fair	yes
youth	medium	yes	excellent	yes
middle	medium	no	excellent	yes
middle	high	yes	fair	yes
senior	medium	no	excellent	no

* CART 알고리즘은 C4.5와 달리 Binary(2진) Split 형태 따름



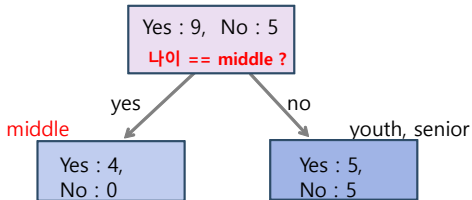
실제 모든 데이터는 정수, 실수 데이터로
처리해야 함
clf =
tree.DecisionTreeClassifier(criterion='
gini',

1) 나이 기준의 각 속성에 대해 분할 후 지니계수 계산



1-1) 자식노드 지니계수(나이 == youth)

$$= \frac{5}{14} * (1 - (\frac{2}{5})^2 - (\frac{3}{5})^2) + \frac{9}{14} * (1 - (\frac{7}{9})^2 - (\frac{2}{9})^2) = 0.394$$



1-2) 자식노드 지니계수(나이 == middle) ★

$$= \frac{4}{14} * (1 - (\frac{4}{4})^2) + \frac{10}{14} * (1 - (\frac{5}{10})^2 - (\frac{5}{10})^2) = 0.357$$

위 동일 방법으로
(나이 == senior) 로 분류

1-3) 자식노드 지니계수(나이 == senior)

$$= \frac{5}{14} * (1 - (\frac{3}{5})^2 - (\frac{2}{5})^2) + \frac{9}{14} * (1 - (\frac{6}{9})^2 - (\frac{3}{9})^2) = 0.457$$

2) 수입 기준의 각 속성에 대해 분할 후 지니계수 계산

앞 나이 기준 분류의 동일 방법으로 수입 기준 속성별 분류

2-1) 자식노드 지니계수(수입 == high)

$$= \frac{4}{14} * (1 - (\frac{2}{5})^2 - (\frac{2}{5})^2) + \frac{10}{14} * (1 - (\frac{3}{10})^2 - (\frac{7}{10})^2) = 0.443$$

2-2) 자식노드 지니계수(수입 == medium)

$$= \frac{4}{14} * (1 - (\frac{1}{5})^2 - (\frac{3}{5})^2) + \frac{10}{14} * (1 - (\frac{4}{10})^2 - (\frac{6}{10})^2) = 0.45$$

2-3) 자식노드 지니계수(수입 == low)

$$= \frac{6}{14} * (1 - (\frac{2}{5})^2 - (\frac{4}{5})^2) + \frac{8}{14} * (1 - (\frac{3}{8})^2 - (\frac{5}{8})^2) = 0.458$$

* 동일 방법으로 다른 특성들에 대한 지니 계수 계산

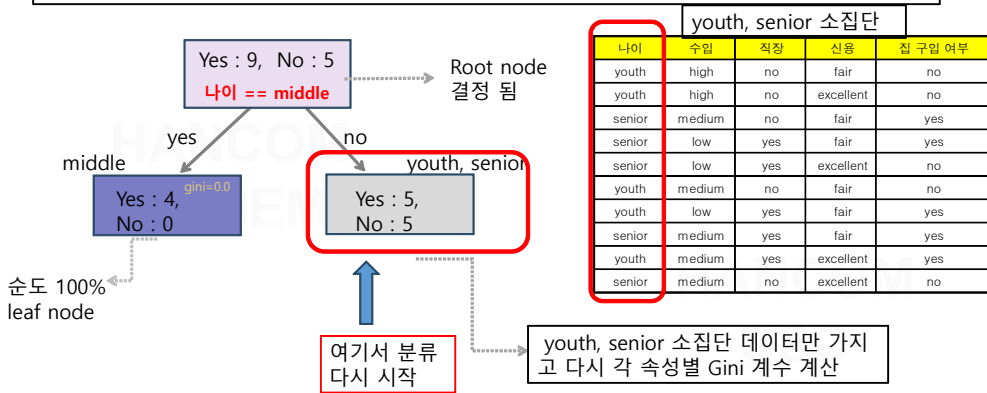
* 각 특성별 자식노드 최소 지니계수

- 자식노드 Gini(나이 == middle) = **0.357**
- 자식노드 Gini(수입 == high) = 0.443
- 자식노드 Gini(신용 ?) = 0.429
- 자식노드 Gini(직장 ?) = 0.367



나이 중 middle 로 분류 했을 경우
정보 이득이 가장 큼,
따라서 (나이 == middle) 기준
으로 가장 먼저 분할

3) 각 terminal node 에 대해 위 지니계수 계산 반복



DecisionTreeClassifier 활용 서울시 행정구역 분류 예

1) 데이터 준비

```
import pandas as pd
```

```
district_dict_list = [  
    {'district': 'Gangseo-gu', 'latitude': 37.551000, 'longitude': 126.849500, 'label': 'Gangseo'},  
    {'district': 'Yangcheon-gu', 'latitude': 37.52424, 'longitude': 126.855396, 'label': 'Gangseo'},  
    {'district': 'Guro-gu', 'latitude': 37.4954, 'longitude': 126.8874, 'label': 'Gangseo'},  
    {'district': 'Geumcheon-gu', 'latitude': 37.4519, 'longitude': 126.9020, 'label': 'Gangseo'},  
    {'district': 'Mapo-gu', 'latitude': 37.560229, 'longitude': 126.908728, 'label': 'Gangseo'},  
  
    {'district': 'Gwanak-gu', 'latitude': 37.487517, 'longitude': 126.915065, 'label': 'Gangnam'},  
    {'district': 'Dongjak-gu', 'latitude': 37.5124, 'longitude': 126.9393, 'label': 'Gangnam'},  
    {'district': 'Seocho-gu', 'latitude': 37.4837, 'longitude': 127.0324, 'label': 'Gangnam'},  
    {'district': 'Gangnam-gu', 'latitude': 37.5172, 'longitude': 127.0473, 'label': 'Gangnam'},  
    {'district': 'Songpa-gu', 'latitude': 37.503510, 'longitude': 127.117898, 'label': 'Gangnam'},  
  
    {'district': 'Yongsan-gu', 'latitude': 37.532561, 'longitude': 127.008605, 'label': 'Gangbuk'},  
    {'district': 'Jongro-gu', 'latitude': 37.5730, 'longitude': 126.9794, 'label': 'Gangbuk'},  
    {'district': 'Seongbuk-gu', 'latitude': 37.603979, 'longitude': 127.056344, 'label': 'Gangbuk'},  
    {'district': 'Nowon-gu', 'latitude': 37.6542, 'longitude': 127.0568, 'label': 'Gangbuk'},  
    {'district': 'Dobong-gu', 'latitude': 37.6688, 'longitude': 127.0471, 'label': 'Gangbuk'},  
  
    {'district': 'Seongdong-gu', 'latitude': 37.557340, 'longitude': 127.041667, 'label': 'Gangdong'},  
    {'district': 'Dongdaemun-gu', 'latitude': 37.575759, 'longitude': 127.025288, 'label': 'Gangdong'},  
    {'district': 'Gwangjin-gu', 'latitude': 37.557562, 'longitude': 127.083467, 'label': 'Gangdong'},  
    {'district': 'Gangdong-gu', 'latitude': 37.554194, 'longitude': 127.151405, 'label': 'Gangdong'},  
    {'district': 'Jungnang-gu', 'latitude': 37.593684, 'longitude': 127.090384, 'label': 'Gangdong'}  
]
```

```
train_df = pd.DataFrame(district_dict_list)  
train_df = train_df[ ['district','longitude','latitude','label'] ] # 훈련데이터 준비  
print(train_df)
```

```

dong_dict_list = [
    {'dong': 'Gaebong-dong', 'latitude': 37.489853, 'longitude': 126.854547, 'label': 'Gangseo'},
    {'dong': 'Gochuk-dong', 'latitude': 37.501394, 'longitude': 126.859245, 'label': 'Gangseo'},
    {'dong': 'Hwagok-dong', 'latitude': 37.537759, 'longitude': 126.847951, 'label': 'Gangseo'},
    {'dong': 'Banghwa-dong', 'latitude': 37.575817, 'longitude': 126.815719, 'label': 'Gangseo'},
    {'dong': 'Sangam-dong', 'latitude': 37.577039, 'longitude': 126.891620, 'label': 'Gangseo'},

    {'dong': 'Nonhyun-dong', 'latitude': 37.508838, 'longitude': 127.030720, 'label': 'Gangnam'},
    {'dong': 'Daechi-dong', 'latitude': 37.501163, 'longitude': 127.057193, 'label': 'Gangnam'},
    {'dong': 'Seocho-dong', 'latitude': 37.486401, 'longitude': 127.018281, 'label': 'Gangnam'},
    {'dong': 'Bangbae-dong', 'latitude': 37.483279, 'longitude': 126.988194, 'label': 'Gangnam'},
    {'dong': 'Dogok-dong', 'latitude': 37.492896, 'longitude': 127.043159, 'label': 'Gangnam'},

    {'dong': 'Pyeongchang-dong', 'latitude': 37.612129, 'longitude': 126.975724, 'label': 'Gangbuk'},
    {'dong': 'Sungbuk-dong', 'latitude': 37.597916, 'longitude': 126.998067, 'label': 'Gangbuk'},
    {'dong': 'Ssangmoon-dong', 'latitude': 37.648094, 'longitude': 127.030421, 'label': 'Gangbuk'},
    {'dong': 'Ui-dong', 'latitude': 37.648446, 'longitude': 127.011396, 'label': 'Gangbuk'},
    {'dong': 'Samcheong-dong', 'latitude': 37.591109, 'longitude': 126.980488, 'label': 'Gangbuk'},

    {'dong': 'Hwayang-dong', 'latitude': 37.544234, 'longitude': 127.071648, 'label': 'Gangdong'},
    {'dong': 'Gui-dong', 'latitude': 37.543757, 'longitude': 127.086803, 'label': 'Gangdong'},
    {'dong': 'Neung-dong', 'latitude': 37.553102, 'longitude': 127.080248, 'label': 'Gangdong'},
    {'dong': 'Amsa-dong', 'latitude': 37.552370, 'longitude': 127.127124, 'label': 'Gangdong'},
    {'dong': 'Chunho-dong', 'latitude': 37.547436, 'longitude': 127.137382, 'label': 'Gangdong'}
]

```

```

test_df = pd.DataFrame(dong_dict_list)
test_df = test_df[ ['dong', 'longitude', 'latitude', 'label'] ] # 테스트 데이터 준비
print(test_df)

```

```
print(train_df['label'].value_counts()) # 5, 5, 5, 5  
print(test_df['label'].value_counts()) # 5, 5, 5, 5
```

```
# train_df 의 longitude(경도 : 세로선), latitude(위도 : 가로선) ==> 학습 데이터  
# train_df 의 label ==> 학습 데이터의 목표(라벨)
```

```
# test_df 의 longitude(경도), latitude(위도) ==> 테스트 데이터  
# test_df 의 label ==> 테스트 데이터의 목표(라벨)
```

```
train_df.drop(['district'], axis=1, inplace=True)  
test_df.drop(['dong'], axis=1, inplace=True)
```

```
X_train = train_df[['longitude','latitude']] # 훈련 데이터 준비  
Y_train = train_df[['label']] # 훈련 타겟 데이터 준비
```

```
# 의사결정트리는 각 특징을 독립적으로 사용하기 때문에 별다른 전처리 과정 필요 없음  
X_test = test_df[['longitude','latitude']] # 테스트 데이터 준비  
Y_test = test_df[['label']] # 테스트 타겟 데이터 준비
```

Gangseo	5
Gangbuk	5
Gangnam	5
Gangdong	5
Name: label, dtype: int64	

Gangseo	5
Gangbuk	5
Gangnam	5
Gangdong	5
Name: label, dtype: int64	

사이킷런 의사결정 트리 모델 학습

```
from sklearn import tree
import numpy as np
import matplotlib.pyplot as plt
from sklearn import preprocessing
```

```
le = preprocessing.LabelEncoder() # 라벨문자를 숫자로 치환
Y_encoded = le.fit_transform(Y_train.values.ravel())
print(Y_encoded) # [3 3 3 3 3 2 2 2 2 2 0 0 0 0 0 1 1 1 1 1]
```

```
print(Y_train.values.ravel())
```

['Gangseo' 'Gangseo' 'Gangseo'
o' 'Gangseo' 'Gangseo' 'Gangnam'
'Gangnam' 'Gangnam' 'Gangnam'
'Gangbuk' 'Gangbuk' 'Gangbuk'
'Gangbuk' 'Gangbuk'
'Gangbuk' 'Gangdong' 'Gangdong'
'Gangdong' 'Gangdong' 'Gangdong'
'Gangdong']

```
print(le.classes_)
```

(알파벳 순으로 다른 클래스 자동 분류)

['Gangbuk' 'Gangdong' 'Gangnam' 'Gangseo']

0 인코딩

1 인코딩

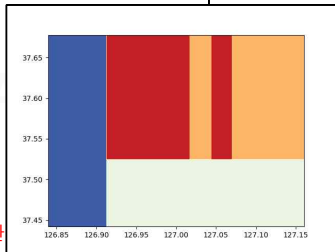
2 인코딩

3 인코딩

```
clf = tree.DecisionTreeClassifier(random_state=35).fit(X_train,Y_encoded) # 모델 학습
```

1) 파라미터 설정 없이 학습한 모델의 과대적합 결정 경계면 시각화

```
def display_decison_surface(clf, X, y):  
    x_min = X['longitude'].min() - 0.01 # 차트 범위가 모든 학습 데이터를 포함 하도록  
    x_max = X['longitude'].max() + 0.01 # 최소, 최대값 기반 그리드 좌표계 생성위해 0.01 씩 + / -  
    y_min = X['latitude'].min() - 0.01  
    y_max = X['latitude'].max() + 0.01  
  
    # meshgrid() :1차원 좌표배열을 2차(N차) 직사각형 그리드 격자점 좌표 생성 함수, 뒷 장표 샘플예제 참조  
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.001),  
                          np.arange(y_min, y_max, 0.001))  
    #print(xx) # [[126.8395 126.8405 126.8415 ... 127.1585 127.1595 127.1605]  
    #print(xx.shape) # (237, 322)  
    # numpy 출력 옵션  
    # precision : default=8 , 소수점 아래를 고정값으로 정하여 출력  
    # threshold: default = 1000, threshold만큼 출력, 더 많을 경우 중략 표현  
    np.set_printoptions(threshold=np.inf)  
    # [ xx yy ] 형태 배열 조합으로 예측 수행  
    Z = clf.predict(np.column_stack([xx.ravel(), yy.ravel()]))  
  
    print(Z.shape) # (76314,)  
  
    Z = Z.reshape(xx.shape) # 예측 데이터 Z 를 xx 또는 yy 차원으로 동일하게 변환  
    # ( 등고색 : 해당 좌표에 동일 높이값을 동일 색으로 출력 ) , 뒷 장표 참조  
    plt.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu)  
    plt.show()  
  
display_decison_surface(clf, X_train, Y_encoded)
```

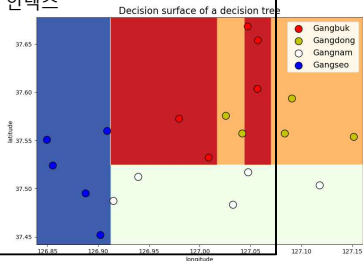


0 (낮은값) 1 3 (높은값)



2) 파라미터 설정 없이 학습한 모델의 과대적합 결정 경계면 및 훈련 데이터 혼합 시각화

```
def display_decison_surface(clf, X, y):  
    ~~~  
    ~~~ # 앞 장표와 동일  
    plt.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu)  
  
    n_classes = len(le.classes_) # 4  
    plot_colors = 'rywb'  
  
    for i, color in zip(range(n_classes), plot_colors):  
        # i : 0, 1, 2, 3  
        # color : r, y, w, b  
        # y : [3 3 3 3 3 2 2 2 2 2 0 0 0 0 0 1 1 1 1 1]  
        idx = np.where(y == i) # np.where : 입력조건을 충족하는 배열의 색인을 생성  
        print(idx) # 해당 idx 값 위치가 강북/강동/강남/강서로 구분된 데이터 인덱스  
        # loc : Fancy Indexing 활용  
        plt.scatter(X.loc[idx,'longitude'],X.loc[idx,'latitude'],  
                    label = le.classes_[i], c=color, edgecolors='black', s=150)  
  
    plt.title('Decision surface of a decision tree', fontsize=16)  
    plt.legend(loc='best',fontsize=14)  
    plt.xlabel('longitude')  
    plt.ylabel('latitude')  
    plt.show()  
  
display_decison_surface(clf, X_train, Y_encoded)
```

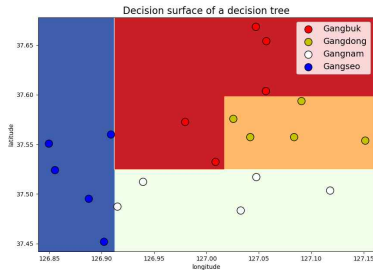


3) 과대적합 회피 파라미터 설정 모델의 결정 경계면 및 훈련 데이터 혼합 시각화

과대적합 회피 파라미터 설정

```
clf = tree.DecisionTreeClassifier(
    max_depth=4,
    min_samples_split=2,
    min_samples_leaf=2,
    random_state=70).fit(X_train,Y_encoded)
```

```
def display_decison_surface(clf, X, y):
    x_min = X['longitude'].min() - 0.01
    x_max = X['longitude'].max() + 0.01
    y_min = X['latitude'].min() - 0.01
    y_max = X['latitude'].max() + 0.01
    ~ ~
    ~ ~ # 앞 장표와 동일
```



파라미터	의미
criterion	노드 분할 결과 품질을 평가하기 위한 기준, 'gini' 와 'entropy' 중 선택, 기본값 'gini'
max_depth	모델(트리)성장 제한 기준, 몇 단계(깊이)까지 성장시킬지 지정
min_samples_split	노드 분할에 필요한 최소 표본수, 이 숫자가 매우 작으면 과대적합 , 반대의 경우 과소적합 발생 기본값 : 2
min_samples_leaf	임시귀(말단) 노드가 되기 위한 최소 표본수, 지정 숫자보다 노드에 더 많은 표본이 있을 경우 노드 분할 가능성이 있음, 기본값 : 1
max_features	최적의 분할을 위해 고려할 최대 feature 개수, Default=None -> 모든 feature 사용
random_state	결과를 고정하기 위해 지정하는 자연수
min_impurity_decrease	여기에 설정한 값 이상으로 불순도가 감소할 경우 노드 분할, 기본값 : 0.0

4) 과대적합 회피 파라미터 설정 모델의 트리구조 시각화

~~

~~ # 앞 장표와 동일

```
le = preprocessing.LabelEncoder()  
Y_encoded = le.fit_transform(Y_train.values.ravel())  
print(Y_encoded)  
print(Y_train.values.ravel())  
print(le.classes_)
```

#과대적합 회피 파라미터 설정 모델

```
clf = tree.DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_split=2,  
    min_samples_leaf=2,  
    random_state=70).fit(X_train,Y_encoded)  
#clf = tree.DecisionTreeClassifier().fit(X_train, Y_encoded) : 과대적합 모델
```

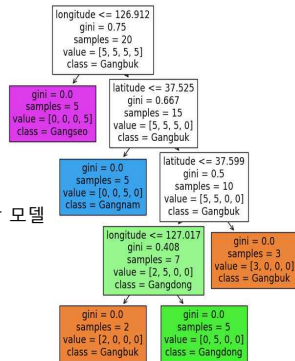
from sklearn.tree import plot_tree

plt.figure(figsize=(8,8))

filled : 클래스에 맞게 노드의 색을 칠함

```
plot_tree(clf, filled=True, feature_names=['longitude','latitude'],  
          class_names=['Gangbuk', 'Gangdong', 'Gangnam', 'Gangseo'])
```

plt.show()



5) 과대적합 회피 파라미터 설정 모델의 임의 데이터 예측 결과 시각화

~~

~~ # 앞 장표와 동일

과대적합 회피 파라미터 설정

```
clf = tree.DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_split=2,  
    min_samples_leaf=2,  
    random_state=70).fit(X_train,Y_encoded)
```

from sklearn.metrics import accuracy_score

```
pred = clf.predict(X_test)
```

```
print('=====  
pred=====')
```

```
print(pred)
```

```
print(le.classes_[pred]) # numpy Fancy Indexing 활용
```

예측정확도

```
print('accuracy : ' + str( accuracy_score(Y_test.values.ravel(), le.classes_[pred]) ) )
```

score : 내부적으로 예측 진행 후 평가 결과 반환 , predict() --> accuracy_score()

Y_test_encoded = le.transform(Y_test.values.ravel())

```
# print('score : ' , clf.score(X_test, Y_test_encoded ) )
```

임의 데이터 예측 및 scatter 출력

```
tempxy = [[127.09, 37.66]]
```

```
temppred = clf.predict( tempxy )
```

```
print(temppred)
```

```
print(le.classes_[temppred])
```

(테스트 데이터 예측)

[3 3 3 3 3 2 2 2 2 2 0 0 0 0 1 1 1 1]

['Gangseo' 'Gangseo' 'Gangseo' 'Gangseo' 'Gangseo' 'Gangnam' 'Gangnam'
'Gangnam' 'Gangnam' 'Gangnam' 'Gangbuk' 'Gangbuk' 'Gangbuk' 'Gangbuk'
'Gangbuk' 'Gangdong' 'Gangdong' 'Gangdong' 'Gangdong' 'Gangdong' 'Gangdong']

accuracy : 1.0

[0]

(임의 데이터 '강북' 으로 분류)

['Gangbuk']

5) 과대적합 회피 파라미터 설정 모델의 임의 데이터 예측 결과 시각화

```
def display_decison_surface(clf, X, tempxy):
    x_min = X['longitude'].min() - 0.01
    x_max = X['longitude'].max() + 0.01
    y_min = X['latitude'].min() - 0.01
    y_max = X['latitude'].max() + 0.01

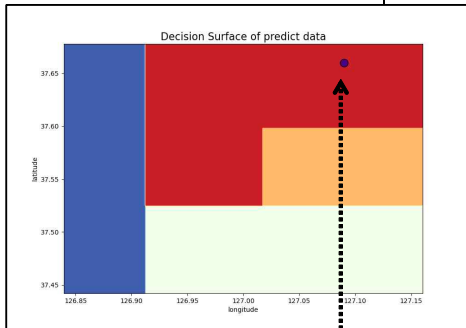
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.001),
                          np.arange(y_min, y_max, 0.001))

    np.set_printoptions(threshold=np.inf)
    Z = clf.predict(np.column_stack([xx.ravel(), yy.ravel()]))
    print(Z.shape)
    Z = Z.reshape(xx.shape)
    plt.figure(figsize=(10,7))
    plt.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu)
```

color 참조 : https://matplotlib.org/stable/gallery/color/named_colors.html
plt.scatter(tempxy[0][0], tempxy[0][1], c='indigo', edgecolors='black', s=150)

```
plt.title('Decision Surface of predict data', fontsize=16)
plt.xlabel('longitude')
plt.ylabel('latitude')
plt.show()
```

```
display_decison_surface(clf, X_train, tempxy)
```



(임의 데이터 '강북' 위치 표시)

matplotlib.pyplot.contourf

- 등고선/색 출력 함수

- contour 와 contourf 는 각각 등고선과 채워진 색을 그림.
- X, Y 파라미터
 - Z에 있는 값의 **좌표**, X,Y 둘다 Z와 동일한 Shape의 2D (np.meshgrid로 생성)
 - 지정하지 않으면 정수 인덱스로 간주
- Z 파라미터
 - 윤곽선이 그려지는 높이 값 (**해당 좌표에 동일 높이값을 동일 색으로 출력**)
- levels 파라미터
 - **등고선/영역의 수와 위치 결정 ,배열과 같은 경우 지정된 수준에서 등고선 그림**
 - 값은 오름차순으로 지정

- colors 파라미터
 - 레벨의 색상, 즉 윤곽선과 윤곽선 영역색, 오름차순으로 레벨에 대해 순환
 - 미 설정시 cmap에 지정된 컬러 맵이 사용
- alpha 파라미터
 - 기본값 : 1 , 0(투명) 과 1(불투명) 사이의 알파 블렌딩 값
- cmap 파라미터
 - 컬러 맵은 레벨 값을 색상에 매핑
 - str 또는 컬러 맵, 기본값 : rcParams ["image.cmap"] (기본값 : 'viridis')
 - 색상과 cmap이 모두 제공되면 오류

샘플 예제)

```
import numpy as np
import matplotlib.pyplot as plt

# linewidth : 1줄당 문자 수(기본값 75)
# threshold : 1개의 배열당 총 element 개수(기본값 1000)
np.set_printoptions(threshold=np.inf, linewidth=np.inf)
x1 = np.linspace(1, 5, 10)
y2 = np.linspace(6,10, 10)
```

[1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5.]

[6. 6.44444444 6.88888889 7.33333333 7.77777778 8.22222222 8.66666667 9.11111111 9.55555556 10.]

샘플 예제)

```
# x1, y2 ==> 배열, 그리드의 좌표를 나타내는 1차원 배열
# 행렬 xx1, yy2를 생성하여 각 행렬의 해당 요소 쌍이 그리드에있는 모든 좌표를 생성(제공)
# x1 내용이 오른쪽으로 그대로 나열되면서, y2 사이즈(수)만큼 아래로 중복 확장
# y2 내용이 아래로 그대로 나열되면서, x1 사이즈(수)만큼 오른쪽으로 중복 확장
xx1, yy2 = np.meshgrid(x1, y2)
print(xx1)
print(yy2)
```

```
[[1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]
 [1. 1.44444444 1.88888889 2.33333333 2.77777778 3.22222222 3.66666667 4.11111111 4.55555556 5. ]]
```

아래로 중복 확장

샘플 예제)

```
# x1, y2 ==> 배열, 그리드의 좌표를 나타내는 1차원 배열
# 행렬 xx1, yy2를 생성하여 각 행렬의 해당 요소 쌍이 그리드에있는 모든 좌표를 생성(제공)
# x1 내용이 오른쪽으로 그대로 나열되면서, y2 사이즈(수)만큼 아래로 중복 확장
# y2 내용이 아래로 그대로 나열되면서, x1 사이즈(수)만큼 오른쪽으로 중복 확장
xx1, yy2 = np.meshgrid(x1, y2)
print(xx1)
print(yy2)
```

```
[[ 6.      6.      6.      6.      6.      6.      6.      6.      6.      6.      ]
 [ 6.44444444 6.44444444 6.44444444 6.44444444 6.44444444 6.44444444 6.44444444 6.44444444 6.44444444 6.44444444]
 [ 6.88888889 6.88888889 6.88888889 6.88888889 6.88888889 6.88888889 6.88888889 6.88888889 6.88888889 6.88888889]
 [ 7.33333333 7.33333333 7.33333333 7.33333333 7.33333333 7.33333333 7.33333333 7.33333333 7.33333333 7.33333333]
 [ 7.77777778 7.77777778 7.77777778 7.77777778 7.77777778 7.77777778 7.77777778 7.77777778 7.77777778 7.77777778]
 [ 8.22222222 8.22222222 8.22222222 8.22222222 8.22222222 8.22222222 8.22222222 8.22222222 8.22222222 8.22222222]
 [ 8.66666667 8.66666667 8.66666667 8.66666667 8.66666667 8.66666667 8.66666667 8.66666667 8.66666667 8.66666667]
 [ 9.11111111 9.11111111 9.11111111 9.11111111 9.11111111 9.11111111 9.11111111 9.11111111 9.11111111 9.11111111]
 [ 9.55555556 9.55555556 9.55555556 9.55555556 9.55555556 9.55555556 9.55555556 9.55555556 9.55555556 9.55555556]
 [10.      10.      10.      10.      10.      10.      10.      10.      10.      10.      ]]
```

오른쪽으로 중복 확장

샘플 예제)

```
Z = xx1**2 + yy2**2 # 좌표에 출력할 Z값 계산
```

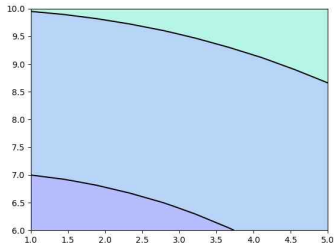
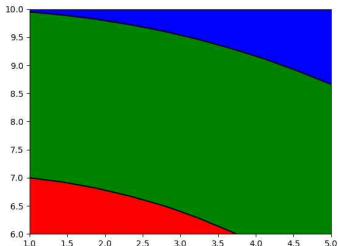
```
# xx1, yy2 좌표에 Z 높이값을 라벨에 매칭하여 등고선 및 색으로 채움
```

```
plt.contour(xx1, yy2, Z, levels=[30,50,100,150], colors = 'black')
```

```
plt.contourf(xx1, yy2, Z, levels=[30,50,100,150], colors = ['r','g','b'])
```

```
#plt.contourf(xx1, yy2, Z, levels=[30,50,100,150], cmap = 'winter', alpha=0.3)
```

```
plt.show()
```



Thank You